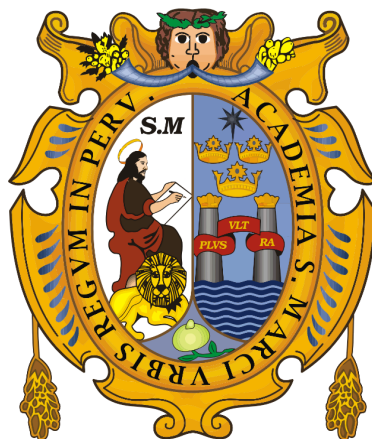


UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

Facultad de Ingeniería de Sistemas e Informática



CURSO: Lenguajes y Compiladores

DOCENTE: Prudencio Vidal, Javier Antonio

GRUPO: 5

INTEGRANTES:

Arboleda Sanchez, Ericka Sofia	24200047
Badillo Castillo, Edwin Piero	24200150
Ccoicca Fernandez, Adrian	24200154
Huamanchahua Huayanay, Oscar	24200161
Maque Espinoza, Steveen Dennys	24200059
Reynoso Castillo, José Luis	24200169
Zarate Vilchez, Axhell Jesus	24200177

Lima, Perú

2026

1. Ejercicio 1 (Pg. 26)

Si $\Sigma = \{a, b, c\}$, $A = \{ba, bc\}$, $B = \{b^n : n \geq 0\}$, entonces hallar AB y BA .

Desarrollo

Como $B = \{b^n : n \geq 0\}$, esto es la potencia de la cadena b .

Como n puede ser igual a 0, el primer elemento es b^0 , lo cual por definición es igual a la cadena vacía λ .

Por lo tanto, podemos visualizar el lenguaje B como: $B = \{\lambda, b, bb, bbb, \dots\}$

Hallando AB

Tomamos la primera cadena de A (ba)

$$ba \cdot \lambda = ba$$

$$ba \cdot b = bab$$

$$ba \cdot bb = babb$$

Por lo tanto esto forma el subconjunto: $\{bab^n : n \geq 0\}$.

Tomamos la segunda cadena de A (bc)

$$bc \cdot \lambda = bc$$

$$bc \cdot b = bcb$$

$$bc \cdot bb = bcbb$$

Por lo tanto esto forma el subconjunto: $\{bcb^n : n \geq 0\}$.

Al unir ambos subconjuntos quedaría:

$$AB = \{bab^n : n \geq 0\} \cup \{bcb^n : n \geq 0\}$$

2. Ejercicio 2 (Pg. 27)

Ejercicio:

Sean

$$A = \{a, \lambda\}, B = \{\lambda\}, C = \{a\}.$$

Hallar:

a. $AB \cap C$

b. $AB \cap AC$

Desarrollo

a. $AB \cap C$

Hallamos AB

Tomamos la primera cadena de A (a) y la concatenamos con B:

$$a \cdot \lambda = a$$

Se forma el subconjunto: $\{a\}$.

Tomamos la segunda cadena de A (λ) y la concatenamos con B:

$$\lambda \cdot \lambda = \lambda$$

Se forma el subconjunto: $\{\lambda\}$.

Al unir ambos subconjuntos quedaría:

$$AB = \{a, \lambda\}$$

Hallamos la intersección con C

$$AB \cap C = \{a\}$$

b. $AB \cap AC$

Como ya tenemos calculado $AB = \{a, \lambda\}$. Tenemos que:

Hallar AC

Tomamos la primera cadena de A (a) y la concatenamos con la cadena de C (a):

$$a \cdot a = aa$$

Se forma el subconjunto: $\{aa\}$.

Tomamos la segunda cadena de A (λ) y la concatenamos con la cadena de C (a):

$$\lambda \cdot a = a$$

Esto forma el subconjunto: $\{a\}$.

Al unir ambos subconjuntos quedaría:

$$AC = \{aa, a\}$$

Finalmente hallamos $AB \cap AC$

$$AB \cap AC = \{a\}$$

$$AC = \{aa, a\}$$

$$AB \cap AC = \{a\}$$

3. Ejercicio 3 (Pg. 28)

Para $\Sigma = \{0, 1\}$, sea $w = 00101110 \in \Sigma^*$.

Desarrollo

Debemos recordar la definición formal: una cadena "v" es subcadena de "w" si existen cadenas "x" e "y" tales que $w = xvy$.

a. ¿1011 es subcadena de w?

Analizamos la cadena original w

$$w = 00 \mathbf{1011} 10$$

Siguiendo la definición formal, existen las subcadenas previas y posteriores donde:

$$x = 00$$

$$v = 1011$$

$$y = 10$$

Al concatenar xvy obtenemos 00101110, que es igual a w. Por lo tanto

1011 si es subcadena de w

b. ¿10 es subcadena de w?

Analizamos la cadena original w

$$w = 00 \mathbf{10} 11 \mathbf{10}$$

La secuencia **10** aparece 2 veces

Primera aparición:

$$x = 00$$

$$v = 10$$

$$y = 1110$$

Segunda aparición:

$$x = 001011$$

$$v = 10$$

$$y = \lambda$$

En ambos casos se cumple la condición $w = xvy$

Por lo tanto **10 es subcadena de w.**

4. Ejercicio 4 (Pg. 33)

- a. Construya un FA para identificadores: un carácter alfabético seguido de hasta 5 caracteres alfanuméricos.

Leyenda del Autómata (Identificadores)

L: Letra (a-z, A-Z)

D: Dígito (0-9)

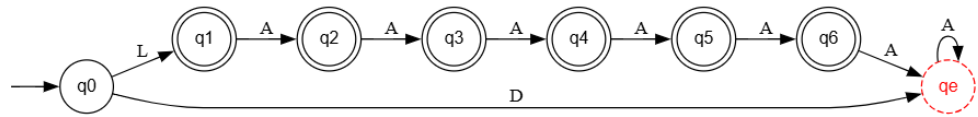
A: Alfanumérico (L o D)

q0: Estado inicial.

q1, q2, q3, q4, q5, q6: Estados de aceptación (identificadores válidos de longitud 1 a 6). Se representan con un **doble círculo**.

qe: Estado de error

Representación Gráfica:



- b. Construya un FA para un comentario de PASCAL: llave abierta {, seguido de cero o más caracteres del conjunto $\Sigma - \}$, seguido de llave cerrada }.

Σ — Alfabeto completo de caracteres

$\Sigma - \}$ — Cualquier carácter excepto }

$\Sigma - \{$ — Cualquier carácter excepto {

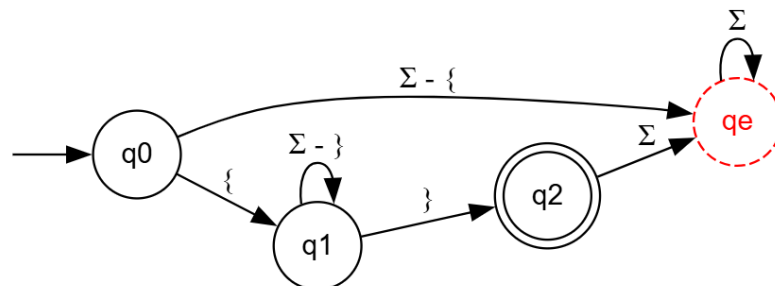
q0 — Estado inicial, espera la llave de apertura {

q1 — Dentro del comentario, consume cualquier carácter excepto }

q2 — Estado de aceptación: comentario correctamente cerrado con }

qe — Estado de error

Representación gráfica



- c. Dibuje el diagrama de transición para reconocer las palabras clave: if, int, for. Combine los tres en un solo FA.

q0 — Estado inicial

q1 — Leído **i**

q2 — Aceptación: reconoció **if**

q3 — Leído **in**

q4 — Aceptación: reconoció **int**

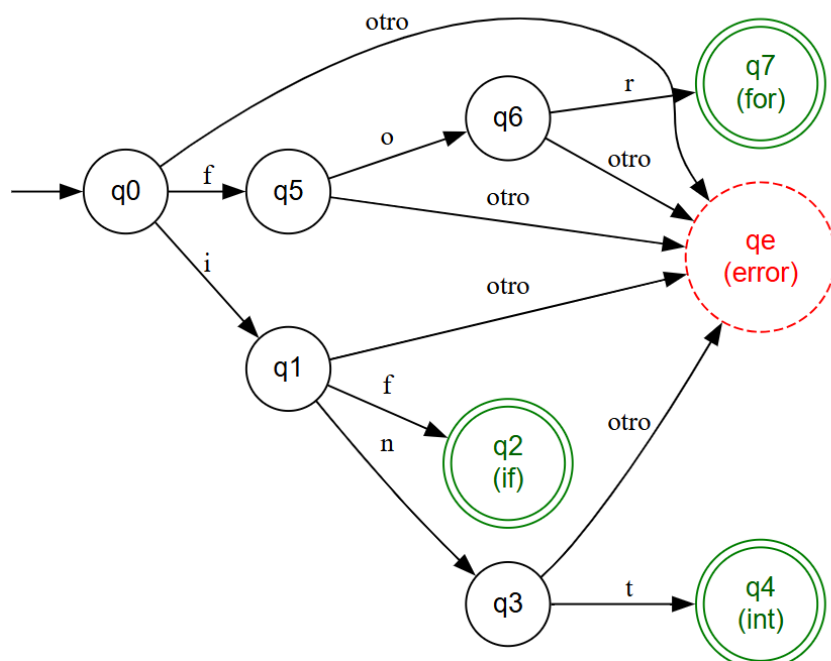
q5 — Leído **f**

q6 — Leído **fo**

q7 — Aceptación: reconoció **for**

qe — Estado de error

Gráfico



d. Complete la tabla de transición δ para su FA del punto 3.

Tabla de transición δ

Estado	i	f	n	t	o	r	otro
q0	q1	q5	qe	qe	qe	qe	qe
q1	qe	q2	q3	qe	qe	qe	qe
*q2	qe	qe	qe	qe	qe	qe	qe
q3	qe	qe	qe	q4	qe	qe	qe
*q4	qe	qe	qe	qe	qe	qe	qe
q5	qe	qe	qe	qe	q6	qe	qe
q6	qe	qe	qe	qe	qe	q7	qe
*q7	qe	qe	qe	qe	qe	qe	qe
qe	qe	qe	qe	qe	qe	qe	qe

(*): Indica los estados de aceptación (q2, q4 y q7)

5. Ejercicio 5 (Pg. 38)

Dibuje el autómata finito determinista que incluya números decimales y enteros con signo o sin signo.

- $\Sigma = \{0,1,2,3,4,5,6,7,8,9,+,-\}$
- $D = 0|1|2|\dots|9$
- $S = \lambda|+| -$
- Expresión regular = $SD^* .? D^+$

Defina formalmente el autómata y dibuje su tabla de transición. Escriba la expresión regular asociada.

Desarrollo:

En el presente ejercicio se construye un Autómata Finito Determinista (DFA) capaz de reconocer números enteros y decimales, considerando además la posible presencia de un signo positivo o negativo. Para ello, se define el alfabeto como $\Sigma = \{0,1,2,3,4,5,6,7,8,9,+,-,.,\}$, donde se incluyen los dígitos, los signos y el punto decimal.

Asimismo, se considera que los dígitos pertenecen al conjunto $D = \{0,1,2,3,4,5,6,7,8,9\}$, mientras que el signo se define como $S = \{+, -, \epsilon\}$, siendo ϵ la ausencia de signo.

El diseño del autómata se basa en la expresión regular:

$$S D^* .? D^+$$

Esta expresión describe la forma general de los números válidos. En términos simples, indica que:

1. El número puede comenzar con un signo (+ o -), o no tenerlo
2. Puede existir una parte entera con cero o más dígitos
3. El punto decimal es opcional
4. Siempre debe haber al menos un dígito al final

Esto permite reconocer cadenas como 123, +45, -78, 12.34 y -0.7. Para construir el DFA, se analiza el proceso de lectura de una cadena paso a paso. La idea es dividir este proceso en etapas, donde cada estado representa lo que se ha leído hasta ese momento. En cada caso se consideran las siguientes situaciones:

1. q_0 : Inicio (aún no se ha leído nada)
2. q_1 : Se ha leído un signo
3. q_2 : Se están leyendo dígitos (parte entera)

4. q_3 : Se ha leído un punto decimal
5. q_4 : Se están leyendo dígitos después del punto
6. q_dead : Se ha producido un error

Estas situaciones dan lugar al conjunto de estados:

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_dead\}$$

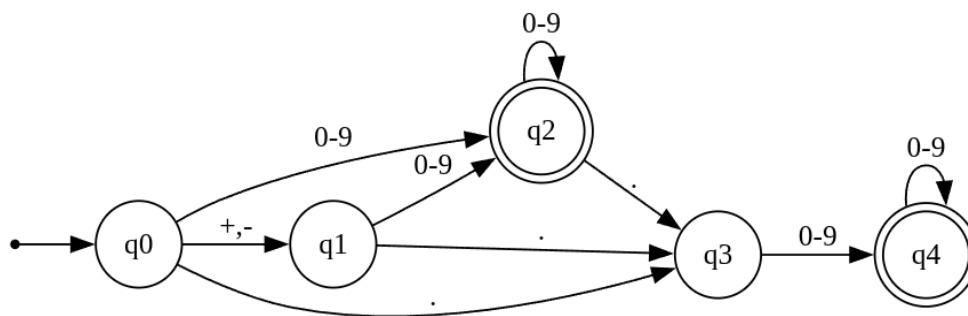
El comportamiento del autómata puede entenderse siguiendo el flujo de lectura:

1. Desde el inicio (q_0):
 - a. Si aparece + o -, se reconoce el signo y se pasa a q_1
 - b. Si aparece un dígito, se inicia directamente el número en q_2
 - c. Si aparece ., se pasa a q_3 (decimal sin parte entera)
2. Desde q_1 (después del signo):
 - a. Si aparece un dígito, se pasa a q_2
 - b. Si aparece ., se pasa a q_3
3. En q_2 (parte entera):
 - a. Se pueden seguir leyendo dígitos indefinidamente
 - b. Si aparece ., se pasa a q_3
 - c. Este estado es de aceptación (número entero válido)
4. En q_3 (después del punto):
 - a. Debe aparecer al menos un dígito
 - b. Se pasa a q_4
5. En q_4 (parte decimal):
 - a. Se pueden seguir leyendo dígitos
 - b. Este estado es de aceptación (número decimal válido)
6. Cualquier transición inválida lleva a q_dead (estado de error)

La función de transición se resume en la siguiente tabla:

Estado	Dígito (0-9)	+	-	.
q0	q2	q1	q1	q3
q1	q2	q_dead	q_dead	q3
q2	q2	q_dead	q_dead	q3
q3	q4	q_dead	q_dead	q_dead
q4	q4	q_dead	q_dead	q_dead
q_dead	q_dead	q_dead	q_dead	q_dead

Con el fin de visualizar el comportamiento del autómata, se presenta a continuación su representación gráfica.



Finalmente, la expresión regular equivalente que describe el lenguaje reconocido por el autómata es:

$$(+ \mid -)? [0 - 9]^* \backslash.? [0 - 9]^+$$

6. Ejercicio 6 (Pg. 40)

Dibuje el autómata finito determinista correspondiente a la siguiente expresión regular:

$(12)^* 01 (0|2)^* 1^*$

Desarrollo:

Análisis de la expresión regular:

El alfabeto de trabajo es $\Sigma = \{0, 1, 2\}$. La expresión regular se descompone en tres partes:

- **$(12)^*$** : Permite cero o más repeticiones de la subcadena “12” al inicio. Esto genera un ciclo de retorno al estado inicial.
- **01**: Secuencia obligatoria. Todo camino de aceptación debe pasar por ella.
- **$(0|2)^*$** : Permite cero o más repeticiones de los símbolos ‘0’ o ‘2’ tras el obligatorio “01”.
- **1^*** : Permite cero o más repeticiones del símbolo ‘1’ al final, exclusivamente.

Definición de estados:

Estado	Descripción
q_0	Estado inicial. Espera el comienzo de un ciclo “12” o el ‘0’ de la secuencia obligatoria.
q_1	Se alcanza tras leer el ‘1’ inicial del ciclo “(12)”. Espera el ‘2’ para volver a q_0 .
q_2	Se alcanza tras leer el ‘0’ obligatorio de “01”. Espera el ‘1’ para completar la secuencia.
q_3	Estado de aceptación. Se alcanza tras completar la secuencia “01”. Acepta ‘0’ y ‘2’ en bucle.
q_4	Estado de aceptación. Se alcanza tras leer al menos un ‘1’ desde q_3 . Acepta ‘1’ en bucle.

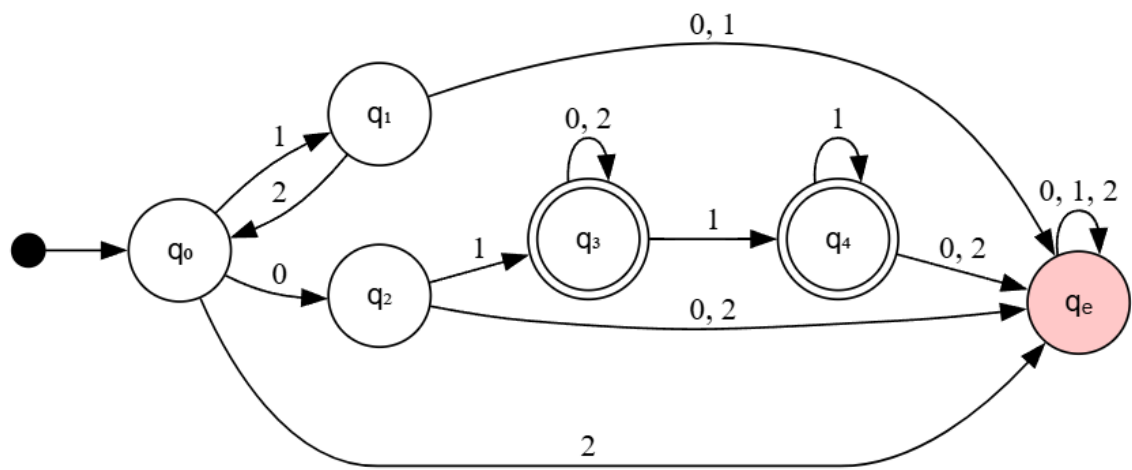
q_e	Estado de error (pozo). Absorbe cualquier transición inválida. No es de aceptación
-------	--

Los estados de aceptación son q_3 y q_4 , ya que tanto la cadena que termina en la secuencia “01” (con cero ‘1’ finales) como la que termina en uno o más ‘1’ son cadenas válidas.

Tabla de transiciones:

Estado	0	1	2	¿Acepta?
$\rightarrow q_0$	q_2	q_1	q_e	No
q_1	q_e	q_e	q_0	No
q_2	q_e	q_3	q_e	No
q_3	q_3	q_4	q_3	Sí
q_4	q_e	q_4	q_e	Sí
q_e	q_e	q_e	q_e	No

Diagrama del DFA:



7. Ejercicio 7 (Pg. 48)

- a. Escriba una RE para identificadores de exactamente 6 caracteres usando solo alternación, concatenación y clausura de Kleene (sin clausura finita).
- b. Escriba una RE para números enteros con signo opcional (+/-), sin ceros a la izquierda.
- c. Escriba una RE para strings de PL/I: comienzan y terminan con ", donde "" representa una comilla literal dentro del string.
- d. Escriba la RE para comentarios multilínea de C/Java (/ * ... */). Dibuje el FA correspondiente.
- e. Para cada RE del punto 1-3, identifique qué operaciones (alternación, concatenación, clausura) se utilizan.

Desarrollo:

- a) Identificadores de exactamente 6 caracteres

Definición de clases:

- L = letra (a-z, A-Z)
- D = dígito (0-9)

Expresión regular: L (L|D) (L|D) (L|D) (L|D) (L|D)

La longitud es exactamente 6 porque se concatenan explícitamente un símbolo L inicial y cinco grupos (L|D), sin usar ninguna clausura que permita longitud variable.

- b) Enteros con signo opcional, sin ceros a la izquierda

Definición de clases:

- S = signo opcional: (+|-| λ)
- N = dígito del 1 al 9

- D = dígito del 0 al 9

Expresión regular: $S (0 \mid N D^*)$

El término **0** cubre el cero simple. El término **N D*** garantiza que todo entero distinto de cero comience con un dígito significativo (1-9), seguido de cero o más dígitos cualquiera, evitando así los ceros a la izquierda.

c) Cadena de PL/I (literales de texto con comillas dobles escapadas)

Definición de clases:

- C = cualquier carácter del alfabeto distinto de la comilla doble (“)

Expresión regular: $" (C \mid "'") "*"$

La expresión admite en el interior de la cadena cualquier número de: caracteres ordinarios (C) o pares de comillas escapadas (""). La cadena comienza y termina con una comilla doble delimitadora.

d) Comentarios multilínea de C/Java ($/* \dots */$)

Expresión regular: $/* ([^*] \mid *+ [^*/])**+ /$

Esta expresión captura el patrón $/* \dots */$ de forma precisa: el interior puede contener cualquier carácter que no sea *, o bien uno o más asteriscos seguidos de un carácter que no sea / (para evitar el cierre prematuro).

Estados del DFA:

Estado	Descripción
q_0	Inicio. Espera el símbolo / de apertura.
q_1	Tras leer /. Espera el * de apertura.
q_2	Dentro del comentario. Lee cualquier carácter excepto *.

q ₃	Dentro del comentario, tras leer al menos un *. Espera / para cerrar.
q ₄	Estado de aceptación. Se alcanza tras leer el / de cierre.

Tabla de transiciones:

Estado	/	*	otro	¿Acepta?
→q ₀	q ₁	—	—	No
q ₁	—	q ₂	—	No
q ₂	q ₂	q ₃	q ₂	No
q ₃	q ₄	q ₃	q ₂	No
q ₄	—	—	—	Sí

e) Identificación de operaciones utilizadas

Inciso	Expresión	Alternación	Concatenación	Clausura de Kleene
(a)	L(L D)(L D)(L D)(L D)(L D)	✓	✓	—
(b)	S(0 ND*)	✓	✓	✓
(c)	"(C "")*"	✓	✓	✓

Observaciones:

- En el inciso (a), la longitud exacta se controla mediante concatenación pura, por lo que no se requiere la clausura de Kleene.

- En los incisos (b) y ©, la clausura de Kleene es esencial: en (b) permite el sufijo numérico de longitud arbitraria, y en © permite el contenido interno de la cadena de longitud variable.

8. Ejercicio 8

Elaborar las expresiones regulares de un Lenguaje de Programación. De preferencia escoger el Lenguaje de Programación del cual diseñarán su compilador y luego obtener los autómatas finitos deterministas. Por ejemplo:

- identificadores: $(_ | [A \dots Z] | [a \dots z]) (_ | [A \dots Z] | [a \dots z] | [0 \dots 9])^*$
- Números enteros sin signo: $0 | [1 \dots 9] [0 \dots 9]^*$
- Números enteros con signo
- Números reales con y sin signo
- Números en notación exponencial: $(0 | [1 \dots 9] [0 \dots 9]^*) (\epsilon | \cdot [0 \dots 9]^*) E (\epsilon | + | -) (0 | [1 \dots 9] [0 \dots 9]^*)$
- Comentarios incluido el $*$:

Desarrollo:

1. Identificadores

En primer lugar, se definen los símbolos permitidos: letras mayúsculas [A-Z], letras minúsculas [a-z] y el guión bajo $_$.

El primer carácter del identificador debe pertenecer a este conjunto, por lo que no se permite repetición en esta posición.

A continuación, pueden aparecer caracteres adicionales que incluyen letras, dígitos [0-9] y el guión bajo. Estos pueden repetirse cualquier número de veces, lo cual se representa mediante el operador * (cero o más repeticiones).

$$(_ | [a - zA - Z]) (_ | [a - zA - Z] | [0 - 9])^*$$

2. Números enteros sin signo

En este caso se consideran dos posibilidades. La primera es el número cero 0. La segunda corresponde a números que no pueden iniciar con cero, por lo que deben comenzar con un dígito del 1 al 9 [1-9], seguido opcionalmente de otros dígitos [0-9].

El operador * indica que estos dígitos adicionales pueden repetirse cero o más veces.

$$0 | [1 - 9][0 - 9]^*$$

3. Números enteros con signo

Se parte de la definición de números enteros sin signo y se incorpora la posibilidad de un signo al inicio. Este signo puede ser + o -, y su presencia es opcional.

El símbolo ? indica que el signo puede aparecer una vez o no aparecer.

$$(+ | -)? (0 | [1 - 9][0 - 9]^*)$$

4. Números reales (con y sin signo)

Un número real se construye a partir de un número entero (con o sin signo), al cual se le puede añadir opcionalmente una parte decimal.

La parte decimal está formada por un punto . seguido de uno o más dígitos $[0-9]^+$. El operador ? indica que esta parte puede existir o no.

$$(+ \mid -)? (0 \mid [1 - 9][0 - 9]^*) (\backslash. [0 - 9]^+)?$$

5. Números con notación exponencial

La notación exponencial se compone de tres partes: un número base (entero o real), el símbolo de exponente E o e, y un exponente entero.

El exponente puede llevar un signo opcional + o -, seguido de uno o más dígitos. Esta estructura permite representar números en forma científica.

$$(+ \mid -)? (0 \mid [1 - 9][0 - 9]^*) (\backslash. [0 - 9]^+)? [E e] (+ \mid -)? (0 \mid [1 - 9][0 - 9]^*)$$

6. Comentarios (incluyendo *)

Los comentarios inician con la secuencia /* y finalizan con */. Entre estos delimitadores puede existir cualquier secuencia de caracteres.

Dado que el símbolo * puede aparecer dentro del comentario, se define una expresión que permite su presencia sin interferir con el cierre del comentario.

$$\backslash \wedge * ([^ \wedge *] \backslash \wedge * + [^ \wedge * /]) * \backslash * + /$$